FUNDAMENTALS OF DATA STRUCTURES --- SAMPLE PAPER 1 --- IDEAL SOLUTION

---- | ---------- |
Initial	[27, 76, 17, 9, 45, 58, 90, 79, 100]
Pass 1	[**9**, 76, 17, 27, 45, 58, 90, 79, 100]
Pass 2	[9, **17**, 76, 27, 45, 58, 90, 79, 100]
Pass 3	[9, 17, **27**, 76, 45, 58, 90, 79, 100]
Pass 4	[9, 17, 27, **45**, 76, 58, 90, 79, 100]
Pass 5	[9, 17, 27, 45, **58**, 76, 90, 79, 100]
Pass 6	[9, 17, 27, 45, 58, **76**, 90, 79, 100]
Pass 7	[9, 17, 27, 45, 58, 76, **79**, 90, 100]
Pass 8	[9, 17, 27, 45, 58, 76, 79, **90**, 100]

Answer: Sorted array --- [9, 17, 27, 45, 58, 76, 79, 90, 100]

Q2) Alternative: Searching and Sorting (OR)

a) Quick sort

Quick sort uses the divide-and-conquer strategy. It selects a pivot, partitions the array around it, and recursively sorts the sub-arrays.

Example: [27, 76, 17, 9, 45, 58, 90, 79, 100] with pivot = last element.

- Partition → elements < 100 on left, > 100 on right
- Recursively sort left partition

Time complexity: Best/Average case O(n log n), Worst case O(n^2).

b) Sentinel search and Indexed sequential search

Sentinel search places the key as the last element (sentinel) to eliminate the boundary check, reducing comparisons to exactly one per iteration.

Indexed sequential search combines sequential access with an index table. The index stores key-pointer pairs for blocks, reducing the search range.

Q3) Linked List

a) Insert into singly linked list

At the beginning:

```
Algorithm: Insert_Begin(head, data)
1. new_node ← allocate memory
2. new_node.data ← data
3. new_node.next ← head
4. head ← new_node
5. return head
```

At the end:

```
Algorithm: Insert_End(head, data)
1. new_node ← allocate memory
2. new_node.data ← data
3. new_node.next ← NULL
4. if head = NULL: return new_node
5. temp ← head
6. while temp.next $\neq $ NULL: temp ← temp.next
7. temp.next ← new_node
8. return head
```

At a given position:

```
Algorithm: Insert_Pos(head, data, pos)
1. if pos = 1: return Insert_Begin(head, data)
2. new_node ← allocate memory; new_node.data ← data
3. temp ← head
4. for i ← 1 to pos-2: temp ← temp.next
5. new_node.next ← temp.next
6. temp.next ← new_node
7. return head
```

b) Polynomial representation using GLL

Generalized Linked List (GLL) represents polynomials with multiple variables efficiently. Each node has three fields: tag, data/pointer, and next.

For polynomial $P(x,y) = 3x^2y + 5xy^2 + 2y$:

- Each term is represented as a node with coefficient, exponent pairs
- GLL allows nested list structure for multi-variable polynomials
- The list head points to the first term node
- Each term node points to the next term node

Thus, GLL provides compact representation for sparse polynomials.

Q5) Stack

a) Stack operations using array

```
Algorithm: Push(S, top, max, item)
1. if top = max-1: print "Stack Overflow"; return
2. top ← top + 1
3. S[top] ← item

Algorithm: Pop(S, top)
1. if top = -1: print "Stack Underflow"; return -1
2. item ← S[top]
3. top ← top - 1
4. return item

Algorithm: Peep(S, top)
1. if top = -1: print "Stack Empty"; return -1
2. return S[top]

Algorithm: IsEmpty(top)
1. return (top = -1)
```

b) Variants of recursion

1. **Direct recursion**: A function calls itself directly. Example: factorial(n) = n × factorial(n−1).
2. **Indirect recursion**: Function A calls function B, which calls function A. Example: is_even(n) calls is_odd(n−1), is_odd(n) calls is_even(n−1).
3. **Tail recursion**: The recursive call is the last statement. Example: print_list(node) where the recursive call is the final operation.
4. **Tree recursion**: Multiple recursive calls within a function. Example: Fibonacci(n) = Fibonacci(n−1) + Fibonacci(n−2).

Thus, understanding recursion variants is essential for algorithm optimization.

Q7) Queue

a) Circular queue using array

```
Algorithm: CQ_Enqueue(Q, front, rear, size, item)
1. if (rear + 1) % size = front: print "Queue Full"; return
2. if front = -1: front ← 0
3. rear ← (rear + 1) % size
4. Q[rear] ← item

Algorithm: CQ_Dequeue(Q, front, rear, size)
1. if front = -1: print "Queue Empty"; return -1
2. item ← Q[front]
3. if front = rear: front ← -1; rear ← -1
4. else: front ← (front + 1) % size
5. return item

Algorithm: CQ_IsEmpty(front)
1. return (front = -1)

Algorithm: CQ_IsFull(front, rear, size)
1. return ((rear + 1) % size = front)
```

Circular queue reuses empty slots by wrapping around, preventing false overflow.

b) Priority queue using array

Priority queue is a data structure where each element has a priority. Higher-priority elements are dequeued before lower-priority ones.

Array implementation:

- Maintain array of elements with priority values
- Enqueue: insert at rear (O(1))
- Dequeue: scan for highest priority element (O(n))
- Two types: **Ascending** (lowest priority first) and **Descending** (highest priority first)

Thus, priority queues are used in CPU scheduling, Dijkstra's algorithm, and Huffman coding.

==========

EXAMINER COMMENTARY

Why this scores full marks:

- Algorithms are presented in clear pseudo-code with numbered steps

- Every numerical pass/step is shown in tabular form
- Technical terms (binary search, pivot, GLL, tail recursion) are bolded
- Array states after each pass of selection sort shown
- Stack operations cover all four basic operations
- Both array and linked implementations covered

Common Deductions:

- Writing pseudo-code without proper indentation or step numbers
- Not showing intermediate passes for sorting algorithms
- Missing edge cases (empty list, full stack) in algorithm pseudo-code
- Confusing circular queue with linear queue
- Not stating time complexity where expected

Time Budget:

- Q1 (18 marks): 42 min → 14 min each for binary search + selection sort working
- Q3 (17 marks): 40 min → 20 min for insert operations + 20 min for GLL
- Q5 (18 marks): 42 min → 20 min for stack + 22 min for recursion
- Q7 (17 marks): 40 min → 20 min for circular queue + 20 min for priority queue

═══